Algorithmic Approaches to Hyperparameter Tuning

April 25, 2026

1 Randome forest

1.1 Exhaustive Grid Search

Grid Search is a brute-force hyperparameter optimization method. It defines a discrete set of values for each hyperparameter and exhaustively evaluates the model's fitness (validation accuracy) for every possible combination in the Cartesian product of these sets.

Algorithm 1 2D Grid Search

Input: Grid boundaries $n_{min}, n_{max}, depth_{min}, depth_{max}$, and step counts $n_{steps}, depth_{steps}$
$N_{grid} \leftarrow \text{linspace}(n_{min}, n_{max}, n_{steps})$
$D_{grid} \leftarrow \text{linspace}(depth_{min}, depth_{max}, depth_{steps})$
$best\_acc \leftarrow -\infty$
$best\_params \leftarrow \emptyset$
for $n \in N_{grid}$ **do**
    for $d \in D_{grid}$ **do**
        $score \leftarrow \text{Evaluate}(n, d)$ ▷ Train RF and calculate validation accuracy
        if $score > best\_acc$ **then**
            $best\_acc \leftarrow score$
            $best\_params \leftarrow \{n, d\}$
        end if
    end for
end for
return $best\_params, best\_acc$

1.2 Genetic Algorithm (GA)

The Genetic Algorithm is an evolutionary metaheuristic inspired by natural selection. The implemented 2D GA maintains a population of candidate solutions (individuals), evaluating their fitness at each generation. It applies Elitism (preserving the best individual), Tournament Selection (size 3) to choose parents, Blend Crossover to generate offspring, and Gaussian Mutation to introduce genetic diversity.

Algorithm 2 Genetic Algorithm for Hyperparameter Tuning

Input: Population size P, Generations G, Mutation rate $P_m = 0.2$, Bounds B
Initialize population $\mathcal{P}_0 \in \mathbb{R}^{P \times 2}$ uniformly within bounds B
$gbest\_score \leftarrow -\infty$
for $t = 1 \rightarrow G$ **do**
    Evaluate fitness $\mathcal{F}$ for each individual $x_i \in \mathcal{P}_{t-1}$
    Update global best $(gbest\_pos, gbest\_score)$ if a better fitness is found
    $\mathcal{P}_t[0] \leftarrow x_{\arg\max(\mathcal{F})}$ ▷ Elitism: Carry over the best individual
    for $i = 1 \rightarrow P - 1$ **do**
        $p_1 \leftarrow \text{TournamentSelection}(\mathcal{P}_{t-1}, \mathcal{F}, \text{size} = 3)$
        $p_2 \leftarrow \text{TournamentSelection}(\mathcal{P}_{t-1}, \mathcal{F}, \text{size} = 3)$
        $\beta \leftarrow U(0,1)^2$ ▷ Blend Crossover
        $child \leftarrow p_1 \cdot \beta + p_2 \cdot (1 - \beta)$
        if $U(0,1) < P_m$ **then** ▷ Gaussian Mutation
            $child[0] \leftarrow child[0] + \mathcal{N}(0, 20)$
            $child[1] \leftarrow child[1] + \mathcal{N}(0, 5)$
        end if
        $\mathcal{P}_t[i] \leftarrow \text{Clip}(child, B)$
    end for
end for
return Round($gbest\_pos$), $gbest\_score$

1.3 Particle Swarm Optimization (PSO)

PSO is a swarm-intelligence algorithm inspired by the flocking behavior of birds. Each candidate solution is a "particle" traversing the search space. A particle's movement is guided by its own best-known position (*pbest*) and the entire swarm's best-known position (*gbest*). The implemented standard PSO uses inertia weight $w = 0.5$ and acceleration coefficients $c_1 = 1.5, c_2 = 1.5$.

Algorithm 3 Particle Swarm Optimization (2D)

Input: Particles $N = 15$, Iterations $I = 20$, $w = 0.5, c_1 = 1.5, c_2 = 1.5$, Bounds B
Initialize positions $X \in \mathbb{R}^{N \times 2}$ uniformly within bounds B
Initialize velocities $V \in \mathbb{R}^{N \times 2}$ uniformly in $[-1, 1]$
$Pbest \leftarrow X$
$Pbest\_scores \leftarrow -\infty \times \mathbf{1}_N$
$Gbest\_pos \leftarrow \emptyset$, $Gbest\_score \leftarrow -\infty$
for $t = 1 \rightarrow I$ **do**
    for $i = 1 \rightarrow N$ **do**
        $score \leftarrow \text{Evaluate}(\text{Round}(X_i))$
        if $score > Pbest\_scores[i]$ **then**
            $Pbest\_scores[i] \leftarrow score$
            $Pbest[i] \leftarrow X_i$
        end if
        if $score > Gbest\_score$ **then**
            $Gbest\_score \leftarrow score$
            $Gbest\_pos \leftarrow X_i$
        end if
    end for
    $R_1, R_2 \sim U(0, 1)^{N \times 2}$
    $V \leftarrow w \cdot V + c_1 \cdot R_1 \cdot (Pbest - X) + c_2 \cdot R_2 \cdot (Gbest\_pos - X)$
    $X \leftarrow \text{Clip}(X + V, B)$
end for
return Round($Gbest\_pos$), $Gbest\_score$

1.4 Optuna Optimization (Bayesian / TPE)

Optuna operates fundamentally differently from GA or PSO by treating hyperparameter tuning as a Sequential Model-Based Optimization (SMBO) problem. By default, it utilizes the Tree-structured Parzen Estimator (TPE) algorithm. Rather than random or heuristic-driven jumps, TPE builds a probabilistic surrogate model of the objective function $P(x|y)$ and $P(y)$, actively sampling the hyperparameter combinations that maximize the Expected Improvement (EI).

Algorithm 4 Optuna Optimization Loop

Input: Number of trials $T = 30$, Parameter search spaces $\mathcal{S}$
Initialize Trial History $\mathcal{H} \leftarrow \emptyset$
for $t = 1 \rightarrow T$ **do**
    $\theta_t \leftarrow \text{TPE\_Suggest}(\mathcal{H}, \mathcal{S})$ ▷ Suggests params maximizing Expected Improvement
    $n\_est \leftarrow \theta_t$['n_estimators']
    $depth \leftarrow \theta_t$['max_depth']
    $min\_split \leftarrow \theta_t$['min_samples_split']
    $score_t \leftarrow \text{Evaluate}(n\_est, depth, min\_split)$
    $\mathcal{H} \leftarrow \mathcal{H} \cup \{(\theta_t, score_t)\}$ ▷ Update the surrogate model with new observation
end for
return $\theta_{best} \in \mathcal{H}$ corresponding to max($score$)